UNIVERSIDAD MIGUEL HERNÁNDEZ DE ELCHE

MÁSTER UNIVERSITARIO EN ROBÓTICA

REDES NEURONALES CONVOLUCIONALES PARA EL ANÁLISIS DE SEÑALES EEG DURANTE LA MARCHA CON UN EXOESQUELETO

TRABAJO FIN DE MÁSTER

Mayo - 2026

AUTOR: Daniel Rosique Egea

DIRECTOR: Eduardo Iañez Martinez

"The human brain, sculpted by millions of years of evolution, is the sole architect of the 'human universe'"

- Miguel Nicolelis

AGRADECIMIENTOS

Agradezco a ...

Gracias a ...

A ... por ...

RESUMEN

- Contexto/problema (1-2 frases)
- Objetivos (1 frase corta)
- Metodología (2-3 frases)
- Resultados ()
  - que el método funciona
  - comparas enfoques
  - cuál es mejor (aunque sea genérico)
- Conclusiones/impacto (1-2 frases)

Se utilizarán redes neuronales convolucionales para detectar patrones cerebrales que detecten la intención de caminar o detenerse con un exoesqueleto. Se ajustarán los modelos para maximizar la detección y reducir los falsos positivos en validación cruzada y en pseudo-online. Los modelos utilizarán información del mismo día, acumulativos o aplicando fine tuning. Se realizará el estudio con registros previamente realizados y se planteará su aplicación a nuevos datos experimentales.

Se utilizarán redes neuronales convolucionales para la detección de patrones cerebrales asociados a la intención de iniciar o detener la marcha en sistemas de asistencia robótica. Este problema resulta clave en el desarrollo de interfaces cerebro-computador aplicadas al control de exoesqueletos, donde la identificación precisa de la intención del usuario es fundamental para garantizar un funcionamiento seguro y eficiente.

El objetivo de este trabajo es analizar el rendimiento de diferentes arquitecturas de redes neuronales convolucionales en la clasificación de señales EEG, así como evaluar distintas estrategias de entrenamiento, incluyendo modelos entrenados con datos del mismo día, enfoques acumulativos y técnicas de fine-tuning. Para ello, se emplean

registros de EEG previamente adquiridos, aplicando esquemas de validación cruzada y evaluaciones en entornos pseudo-online que simulan condiciones reales de uso.

Los resultados obtenidos permiten comparar el impacto de la arquitectura, el método de entrenamiento y la variabilidad entre usuarios en el rendimiento del sistema, destacando aquellas configuraciones que maximizan la precisión y reducen los falsos positivos. Asimismo, se identifican diferencias significativas entre los distintos enfoques evaluados, lo que proporciona información relevante para el diseño de sistemas más robustos.

Finalmente, este trabajo sienta las bases para la aplicación de estos modelos en escenarios reales, contribuyendo al desarrollo de sistemas de control más fiables en exoesqueletos y otras tecnologías de asistencia basadas en interfaces cerebro-computador.

ÍNDICE

AGRADECIMIENTOS iii

RESUMEN v

ÍNDICE DE TABLAS xiii

ÍNDICE DE FIGURAS xiv

ÍNDICE DE CÓDIGOS xv

1. Introducción 1

1.1. Motivación 1

1.2. Definición 2

1.3. Alternativas Consideradas 2

1.4. Objetivos a Conseguir 3

1.4.1. Objetivos específicos 3

2. Estado del Arte 4

2.1. Conceptos Relevantes del Dominio 4

2.2. Relación con Proyectos Similares 5

2.2.1. Métodos clásicos 5

2.2.2. Deep Learning aplicado a EEG 5

2.2.3. Aplicaciones en exoesqueletos / rehabilitación 5

2.2.4. Limitaciones actuales 6

2.2.5. Tu hueco (esto es clave) 6

3. Material y Métodos **7**

3.1. Vision General del Sistema 7

3.2. Materiales 9

3.2.1. Hardware 9

3.2.2. Sofware 9

3.3. Estructura Dataset 11

3.4. Preprocesamiento de datos 11

3.5. Modelos utilizados 11

3.5.1. EEGNet 12

3.5.2. DeepConvNet 12

3.5.3. ShallowConvNet 12

3.6. Metodología experimental 12

3.6.1. Variables del estudio 12

3.6.2. Estrategia de entrenamiento 13

3.6.3. Evaluación 13

3.7. Infraestructura Experimental 13

3.7.1. Diseño de experimentos 14

4. Resultados y Discusión **15**

4.1. Tarea Estática 15

4.1.1. Comparación de Arquitecturas 15

4.1.2. Comparación de Método de Entrenamiento 15

4.1.3. Variabilidad Entre Usuarios 15

4.1.4. Mejor Configuración 15

4.2. Tarea Movimiento 15

4.2.1. Comparación de Arquitecturas 16

4.2.2. Comparación de Método de Entrenamiento 16

4.2.3. Variabilidad Entre Usuarios 16

4.2.4. Mejor Configuración 16

4.3. Hiperparametros 16

5. Conclusiones **17**

6. Contenido de TFG y TFM **18**

6.0.1. Estructura y contenidos de un Trabajo de Investigación 18

7. TABLAS **19**

7.1. Posicionamiento 20

7.2. Entorno \`longtable` 21

8. IMÁGENES **24**

9. CÓDIGO **25**

BIBLIOGRAFÍA **29**

ANEXOS **30**

A. Primer anexo 30

B. Segundo anexo 31

ÍNDICE DE TABLAS

7.1. Cinco primeros términos de la sucesión de Fibonacci 19

7.2. Cincuenta primeros términos de la sucesión de Fibonacci 22

ÍNDICE DE FIGURAS

3.1. Pipeline experimental 8

ÍNDICE DE CÓDIGOS

9.1. Código utilizado para generar la gráfica **??** 26

1. Introducción

Aquí haces el contexto global contendr´a un estudio del estado actual de la t´ecnica sobre el problema plateado, las alternativas consideradas y los objetivos a conseguir.

- Qué es la neurorrobótica / BCI
- Qué son las señales EEG (muy breve)
- Aplicación: exoesqueletos / rehabilitación
- Problema clave: detectar intención de movimiento

Sin embargo, este problema presenta múltiples desafíos...

1.1. Motivación

Aquí vendes por qué esto importa

- Problemas reales:
  - pacientes con movilidad reducida
  - necesidad de control natural
- Por qué EEG:
  - no invasivo
  - accesible
- Problema:
  - ruido
  - variabilidad entre usuarios
  - falsos positivos (MUY importante en exoesqueletos)

Esto justifica la necesidad de métodos robustos. . .

1.2. Definición

- Qué señal tienes:
  - EEG
- Qué quieres detectar:
  - intención de caminar
  - parada
- Qué tipo de problema es:
  - clasificación binaria (o multiclase si aplica)
- Contexto:
  - entorno pseudo-online
  - datos offline

Muy importante: que quede claro qué entra y qué sale del sistema

1.3. Alternativas Consideradas

Aquí haces un mini "estado del arte light"

Qué meter:

- Métodos clásicos:
  - features + SVM / LDA
- Métodos deep learning:
  - CNNs (EEGNet, DeepConvNet...)
- Diferentes enfoques de entrenamiento:
  - por usuario

  - generalizados
  - fine-tuning

No te líes mucho, esto no es el estado del arte completo Termina con:

"En este trabajo se opta por..."

1.4. Objetivos a Conseguir

Aquí tienes que ser cristalino

Objetivo general (1 frase) Desarrollar/evaluar modelos CNN para detectar intención de marcha en EEG

1.4.1. Objetivos específicos

Algo tipo:

- Evaluar diferentes arquitecturas (EEGNet, DeepConvNet...)
- Comparar métodos de entrenamiento:
  - same-day
  - acumulativo
  - fine-tuning
- Analizar variabilidad entre usuarios
- Evaluar rendimiento en:
  - validación cruzada
  - pseudo-online
- Reducir falsos positivos

Esto suele gustar MUCHO a tribunales

2. Estado del Arte

2.1. Conceptos Relevantes del Dominio

EEG

- Qué es (muy breve)
- Qué mide (actividad eléctrica cerebral)
- Problemas:
- ruido
- baja resolución espacial
- variabilidad entre sujetos

BCI

- Qué es
- Pipeline típico:
  - adquisición → preprocesado → clasificación → acción

Aquí puedes conectar con tu caso (exoesqueleto)

Intención de movimiento

- Qué es detectar intención
- Tipos de señales:
  - motor imagery
  - movimiento real
- Importancia en control de dispositivos

Deep Learning en EEG

- Por qué CNNs
- Ventajas vs métodos clásicos:
  - menos feature engineering
  - mejor captura de patrones espaciales/temporales

2.2. Relación con Proyectos Similares

Estructura muy clara:

2.2.1. Métodos clásicos

- LDA, SVM
- Features manuales
- Limitaciones

2.2.2. Deep Learning aplicado a EEG

CNNs (EEGNet, DeepConvNet...)

Qué han conseguido

Problemas abiertos:

generalización

dependencia del usuario

2.2.3. Aplicaciones en exoesqueletos / rehabilitación

Uso de BCI para control

Problema clave:

detección fiable de intención

2.2.4. Limitaciones actuales

Aquí es donde te preparas el terreno:

variabilidad entre sujetos

necesidad de recalibración

falsos positivos peligrosos

falta de evaluación en escenarios realistas

2.2.5. Tu hueco (esto es clave)

Termina con algo así:

"En este contexto, este trabajo se centra en…"

Aquí conectas directamente con tu TFM:

comparación de métodos de entrenamiento

evaluación pseudo-online

análisis sistemático

3. Material y Métodos

3.1. Vision General del Sistema

Se ha definido un pipeline de ejecución que permite realizar múltiples experimentos de forma iterativa y automatizada. Este pipeline está compuesto por distintos módulos encargados de gestionar el preprocesamiento de los datos, la configuración de los modelos, el entrenamiento y la evaluación de los mismos.

Con el objetivo de facilitar la reproducibilidad y la exploración sistemática de parámetros, el código se ha estructurado como un microframework que permite modificar de forma sencilla las variables de estudio, así como controlar la ejecución de los experimentos. Este enfoque ha resultado especialmente útil dado el elevado número de ejecuciones.

Para organizar la ejecución, se ha definido una estructura jerárquica en tres niveles:

- **Experimento**: nivel superior que agrupa el conjunto completo de configuraciones evaluadas. Un experimento consiste en la ejecución sistemática de múltiples combinaciones de parámetros.
- **Iteración**: cada una de las ejecuciones individuales del pipeline dentro de un experimento, asociada a una combinación específica de parámetros (arquitectura, método de entrenamiento y tipo de tarea).
- **Entrenamiento**: nivel más bajo de la jerarquía, correspondiente al entrenamiento y evaluación de un modelo para un usuario y sesión concretos dentro de una iteración.

De este modo, cada experimento está compuesto por múltiples iteraciones, y cada iteración incluye varios entrenamientos, uno por cada usuario y sesión del dataset. Los experimentos se definen mediante la combinación de distintos parámetros, entre los que destacan el tipo de tarea, la arquitectura del modelo y el método de entrenamiento. Para cada combinación se ejecuta una iteración independiente

En cada iteración, el sistema inicializa los parámetros de entrenamiento y carga el conjunto de datos correspondiente al tipo de tarea. A continuación, se ejecuta el método de entrenamiento seleccionado, en el que se itera sobre los distintos usuarios y sesiones disponibles en el dataset. Para cada caso, se inicializa el modelo, se realiza el entrenamiento y se evalúa su rendimiento.

Finalmente, los resultados obtenidos, junto con los parámetros utilizados, se almacenan de forma estructurada, permitiendo su posterior análisis.

Este proceso permite realizar un estudio posterior de los resultados obtenidos por cada combinación de parámetros definida.

La Figura 3.1 resume la organización jerárquica del pipeline experimental.

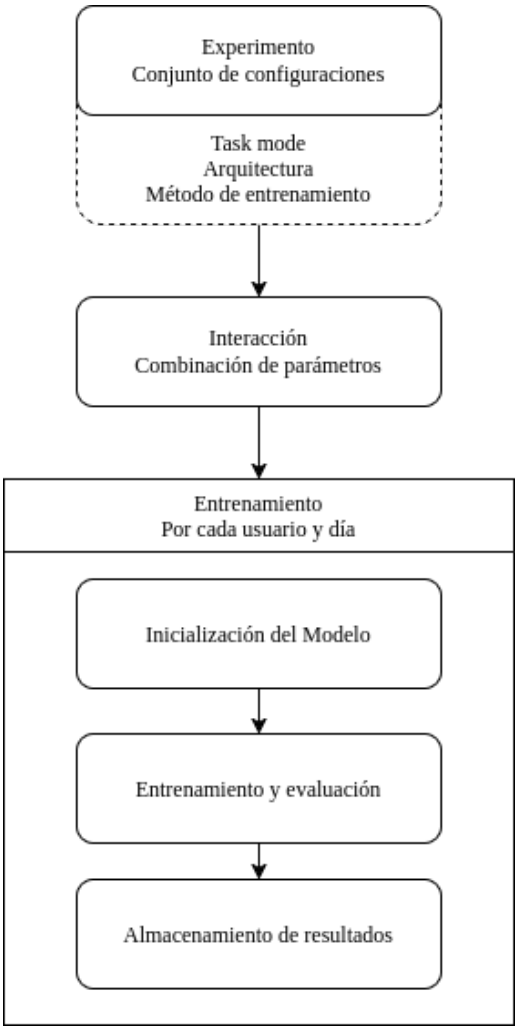

Figura 3.1: Pipeline experimental

3.2. Materiales

3.2.1. Hardware

Para ejecutar los distintos entrenamientos se ha utilizado un ordenador con las siguientes características:

- **CPU**: Intel Core i9-14900HX
- **RAM**: 31 GiB
- **GPU**: NVIDIA GeForce RTX 4060
- **GPU RAM**: 8 GiB

Estas características han resultado suficientes para completar los experimentos planteados sin limitaciones críticas. El tiempo medio de entrenamiento por iteración ha sido aproximadamente 40 minutos, dependiendo de la configuración del modelo y de los parámetros de entrenamiento.

El tiempo total de cómputo se estima mediante:

$$\text{Tiempo total (h)} = \frac{N_{\text{iteraciones}} \times t_{\text{iteración}}}{60}$$

Donde $N_{\text{iteraciones}} = 1917$ y $t_{\text{iteración}} = 40$minutos, resultando en un tiempo total aproximado de 766.8 horas ($\approx$ 31días).

El uso de hardware con mayores prestaciones permitiría reducir significativamente el tiempo total de entrenamiento y facilitar la realización de un mayor número de experimentos.

3.2.2. Sofware

Para el desarrollo del proyecto se ha utilizado una infraestructura basada en herramientas ampliamente empleadas en el ámbito del aprendizaje automático y la

ingeniería de software.

Como lenguaje de programación principal se ha utilizado Python[1], debido a su amplio ecosistema de librerías científicas y de aprendizaje automático, así como su facilidad de integración con frameworks de alto nivel.

Para la construcción y entrenamiento de los modelos se han utilizado TensorFlow[2] y Keras[3], frameworks de aprendizaje profundo que permiten definir, entrenar y evaluar redes neuronales de forma eficiente. Keras se ha empleado como interfaz de alto nivel sobre TensorFlow, facilitando la experimentación rápida con distintas arquitecturas.

Durante las etapas iniciales del proyecto se empleó Jupyter Notebook[4] para la realización de pruebas exploratorias y la visualización preliminar de resultados. Sin embargo, su uso fue descartado en fases posteriores debido a la dificultad para mantener una estructura de código modular y escalable en experimentos de mayor complejidad.

Posteriormente, se adoptó Visual Studio Code[5] como entorno de desarrollo principal, debido a su capacidad para gestionar múltiples tipos de archivos y su integración con herramientas de desarrollo. Se utilizaron extensiones específicas para Python, Jupyter y GitHub, permitiendo un flujo de trabajo unificado.

Como sistema de control de versiones se ha utilizado GitHub[6], que facilita la gestión del código, el seguimiento de cambios y la colaboración con el tutor del proyecto. Además, permite la sincronización del repositorio entre distintos dispositivos de forma eficiente.

Para la visualización de resultados se ha utilizado Vega-Altair[7], una librería de visualización estadística declarativa enfocada en la simplicidad, eficiencia y legibilidad. A diferencia de enfoques imperativos como Matplotlib, Altair permite definir visualizaciones de forma estructurada y reproducible, enfocándose en lo que se quiere

[1] https://www.python.org/
[2] https://www.tensorflow.org/
[3] https://keras.io/
[4] https://jupyter.org/
[5] https://code.visualstudio.com/
[6] https://github.com/
[7] https://altair-viz.github.io/

observar en lugar de en los detalles de la implementación. Esto conduce a un código más limpio y legible[3].

Como gestor de base de datos se ha utilizado DuckDB[8], un sistema de bases de datos analítico embebido que permite realizar consultas SQL de forma eficiente directamente sobre archivos locales. Su integración con Python y su alto rendimiento lo hacen especialmente adecuado para el procesamiento de grandes volúmenes de resultados experimentales[5].

Todas estas herramientas han sido integradas en un flujo de trabajo unificado orientado a la ejecución reproducible de experimentos.

3.3. Estructura Dataset

3.4. Preprocesamiento de datos

la Pipeline qué hace tu sistema de principio a fin

Ejemplo:

entrada: EEG

procesamiento

modelo

salida: predicción

3.5. Modelos utilizados

Aquí metes:

[8] https://duckdb.org/

3.5.1. EEGNet

3.5.2. DeepConvNet

3.5.3. ShallowConvNet

(si lo usas)

Para cada uno:

breve descripción (2–3 líneas)

por qué lo eliges

NO pongas la arquitectura entera en detalle aquí (eso es más de anexos si acaso)

3.6. Metodología experimental

Aquí es donde entra todo tu proyecto de verdad.

3.6.1. Variables del estudio

Muy importante estructurarlo así:

Arquitectura del modelo

Método de entrenamiento:

same-day

acumulativo

fine-tuning

Tarea:

static / motion

Usuario

3.6.2. Estrategia de entrenamiento

epochs

batch size

callbacks:

early stopping

reduce LR

función de pérdida

métrica (accuracy)

No te líes demasiado, pero deja claro cómo entrenas

3.6.3. Evaluación

MUY importante:

validación cruzada

pseudo-online

explica qué significa pseudo-online (aunque sea breve)

3.7. Infraestructura Experimental

Aquí metes tu microframework (esto es TU aporte técnico fuerte)

sistema para ejecutar múltiples experimentos gestión de parámetros almacenamiento de resultados (DuckDB) automatización

Esto es diferencial → véndelo bien

Se ha programado un microframework para el desarrollo de experimentos de Machine Learning.

Como se ha comentado en el apartado **??**, se ha realizado la comparación de tres modelos de redes neuronales convolucionales para la clasificación de la intención de caminar o detenerse con un exoesqueleto. Se ha utilizado un dataset de EEG con 27 canales, obtenido de una muestra de un estudio previo XXXXXX. Los parametros a comparar son la arquitectura de la red, el metodo de entrenamiento, la tarea a predecir y la variabilidad entre usuarios.

Al necesitarse de cientos de ejecuciones para obtener todos los datos necesarios, durante las etapas iniciales se tomó la decisión de enfocar el proyecto de manera horizontal, preparando una base de ejecución sobre la cual modificar facilmente todo lo necesario para cada ejecución y permitiendo ejecutar multiples experimentos de manera secuencial.

(((((((Aquí no sé si ir diicendo TOOOOODo lo que he ido poniendo en el código. Puedo ir siguiendo el timeline de Github y hacer un resumen de los commits)))))))

3.7.1. Diseño de experimentos

Aquí explicas cómo organizaste TODO

combinaciones de parámetros nº de ejecuciones cómo comparas resultados

Esto conecta directamente con Results luego

4. Resultados y Discusión

se describir´an y analizar´an los resultados obtenidos siguiendo la metodolog´ıa propuesta.

4.1. Tarea Estática

4.1.1. Comparación de Arquitecturas

Me da igual todo, solo me centro en los modelos. hago avg y std con los metodos y los usuarios

4.1.2. Comparación de Método de Entrenamiento

Me da igual todo, solo me centro en los metodos. hago avg y std con los modelos y los usuarios

4.1.3. Variabilidad Entre Usuarios

Me da igual todo, solo me centro en los usuarios. hago avg y std con los metodos y los modelos ç

4.1.4. Mejor Configuración

De aquí sacaría algo rollo: La mejor combinación en cuanto a usuario+metodo+modelo –¿ej (gráfica sólamente de usuairo1+entrenamientoA+eegnet)

4.2. Tarea Movimiento

mismo esquema

4.2.1. Comparación de Arquitecturas

4.2.2. Comparación de Método de Entrenamiento

4.2.3. Variabilidad Entre Usuarios

4.2.4. Mejor Configuración

4.3. Hiperparametros

Analysis EEGNet hyperparameters comparison plots

5. Conclusiones

en funci´on de los resultados obtenidos, se expondr´an las con-clusiones de la investigaci´on y se propondr´an trabajos futuros.

6. Contenido de TFG y TFM

6.0.1. Estructura y contenidos de un Trabajo de Investigación

Un TFG o TFM en la modalidad de Trabajo de Investigación deberá estructurarse en los siguientes apartados:

1. *Introducción*: contendrá un estudio del estado actual de la técnica sobre el problema plateado, las alternativas consideradas y los objetivos a conseguir.
2. *Material y métodos*: se describirán los equipos y materiales utilizados para su realización, la metodología empleada y una descripción completa de los experimentos realizados.
3. *Resultados y discusión*: se describirán y analizarán los resultados obtenidos siguiendo la metodología propuesta.
4. *Conclusiones*: en función de los resultados obtenidos, se expondrán las conclusiones de la investigación y se propondrán trabajos futuros.
5. *Anexos*: tantos como se considere necesario en función de los aspectos concretos que se quieran especificar.
6. *Bibliografía*: se indicarán todas las referencias bibliográficas utilizadas, ordenadas con código para su cita en la memoria.

7. TABLAS

Las tablas se definen en el entorno `table`. Existen infinidad de posibilidades en cuanto a su formato: omitir o dibujar líneas horizontales y verticales, fusionar columnas y filas, alinear el contenido a la derecha, izquierda o centro, y demás opciones resumidas en `https://www.overleaf.com/learn/latex/tables`. Dado que la forma de construir una tabla directamente en código LaTeX está lejos de ser cómoda e intuitiva, quizás lo más recomendable sea acudir a editores de tablas que generan automáticamente el código correspondiente y cuya interfaz es similar a la que puede encontrarse en Excel, como por ejemplo `https://www.tablesgenerator.com/`. Un ejemplo sencillo de tabla se muestra a continuación:

$\boldsymbol{n}$	$\boldsymbol{a_n}$	$\boldsymbol{a_{n+1}}$	$\boldsymbol{\varphi}$ $(=a_{n+1}/a_n)$
1	1	1	1
2	1	2	2
3	2	3	1,5
4	3	5	1,66666667
5	5	8	1,6
6	8	13	1,625
7	13	21	1,61538462
8	21	34	1,61904762
9	34	55	1,61764706
10	55	-	-

Tabla 7.1: Cinco primeros términos de la sucesión de Fibonacci

El título de la tabla se indica mediante el comando `\caption` (este comando no solamente sirve para añadir un título a la tabla, sino que es esencial para que ésta aparezca en el índice de tablas), y, al igual que en el caso de los títulos de capítulos (ver apartado **??**), es muy recomendable añadir el comando `\label` para poder referirse a la tabla en cuestión en partes posteriores (o anteriores) del documento.

7.1. Posicionamiento

Las tablas (al igual que otros elementos como imágenes o diagramas, como se verá más adelante) pueden posicionarse en distintos lugares de la página y en distintas posiciones con respecto al texto. La forma más común de situar una tabla es inmediatamente después de un párrafo y centrada en la página (como en el caso de la tabla 7.1), lo que se consigue indicando `[H]` al iniciar el entorno `table` y añadiendo el comando `\centering`, respectivamente. Una guía que recopila las distintas opciones en lo que se refiere al posicionamiento de tablas e imágenes puede consultarse en `https://www.overleaf.com/learn/latex/positioning_images_and_tables`.

7.2. Entorno \longtable

En el caso de que una tabla sea demasiado larga como para caber en una única página se puede utilizar el entorno `longtable`, mediante el cual LaTeX secciona la tabla de forma automática en tantas partes como sea necesario.

n	a_n	a_{n+1}	φ $(=a_{n+1}/a_n)$
1	1	1	1
2	1	2	2
3	2	3	1,5
4	3	5	1,66666667
5	5	8	1,6
6	8	13	1,625
7	13	21	1,61538462
8	21	34	1,61904762
9	34	55	1,61764706
10	55	89	1,61818182
11	89	144	1,61797753
12	144	233	1,61805556
13	233	377	1,61802575
14	377	610	1,61803714
15	610	987	1,61803279
16	987	1597	1,61803445
17	1597	2584	1,61803381
18	2584	4181	1,61803406
19	4181	6765	1,61803396
20	6765	10946	1,618034
21	10946	17711	1,61803399
22	17711	28657	1,61803399
23	28657	46368	1,61803399
24	46368	75025	1,61803399

n	a_n	a_{n+1}	φ $(=a_{n+1}/a_n)$
25	75025	121393	1,61803399
26	121393	196418	1,61803399
27	196418	317811	1,61803399
28	317811	514229	1,61803399
29	514229	832040	1,61803399
30	832040	1346269	1,61803399
31	1346269	2178309	1,61803399
32	2178309	3524578	1,61803399
33	3524578	5702887	1,61803399
34	5702887	9227465	1,61803399
35	9227465	14930352	1,61803399
36	14930352	24157817	1,61803399
37	24157817	39088169	1,61803399
38	39088169	63245986	1,61803399
39	63245986	102334155	1,61803399
40	102334155	165580141	1,61803399
41	165580141	267914296	1,61803399
42	267914296	433494437	1,61803399
43	433494437	701408733	1,61803399
44	701408733	1134903170	1,61803399
45	1134903170	1836311903	1,61803399
46	1836311903	2971215073	1,61803399
47	2971215073	4807526976	1,61803399
48	4807526976	7778742049	1,61803399
49	7778742049	1,2586E+10	1,61803399
50	1,2586E+10	-	-

Tabla 7.2: Cincuenta primeros términos de la sucesión de Fibonacci

Existen distintas alternativas en cuanto a qué elementos incluir tanto en la primera como en la última línea de cada sección de tabla (en el caso de la tabla 7.2 se ha

elegido repetir la primera línea en cada una de sus secciones), que pueden consultarse en `https://texblog.org/2011/05/15/multi-page-tables-using-longtable/`.

8. IMÁGENES

Las imágenes se insertan mediante el comando `\includegraphics` que es conveniente situar en el entorno `figure` (mismo entorno utilizado, como se verá más adelante, para gráficas o diagramas). Un ejemplo de imagen se muestra a continuación (al utilizar Overleaf es esencial cargar la imagen en el directorio de trabajo antes de insertarla en el documento):

En la inmensa mayoría de casos el tamaño original de la imagen no se adapta correctamente a las dimensiones de la página, por lo que es necesario redimensionar la imagen mediante el argumento `scale` de `\includegraphics`.

Al igual que para las tablas, existen distintas alternativas en cuanto a su posicionamiento (que, se recuerda, pueden consultarse en `https://www.overleaf.com/learn/latex/positioning_images_and_tables`), el título se indica mediante el comando `\caption` y la referencia mediante el comando `\label`.

Existen, además, diversas opciones en lo relativo al manejo de imágenes que no se detallan en este documento, pero que pueden consultarse en `https://es.overleaf.com/learn/latex/Inserting_Images`

9. CÓDIGO

Los fragmentos de código se incorporan al informe en el entorno `code` y corresponden al paquete `listings`. A continuación se presenta el fragmento de código Python que genera la figura **??** introducida en el apartado **??**:

```python
# Se importa las librerías matplotlib y numpy:
import matplotlib
import matplotlib.pyplot as plt
import numpy as np

# Se configura matplotlib para exportar la gráfica a ".pgf":
matplotlib.use("pgf")
matplotlib.rcParams.update({
    "pgf.texsystem": "pdflatex",
    "font.family": "serif",
    "text.usetex": True,
    "pgf.rcfonts": False,
})

# Se define el dominio de la variable temporal y el "sample rate":
t = np.linspace(0, 6, 1000, endpoint=True)

# Se representan los ocho armónicos de la onda cuadrada:
for k in range(1,9):
    if k == 1:
        plt.plot(t, (8/np.pi)*np.sin(2*np.pi*(2*k-1)*0.5*t)/(2*k-1), linewidth=0.5, color="gray", label="Ondas sinusoidales")
    else:
        plt.plot(t, (8/np.pi)*np.sin(2*np.pi*(2*k-1)*0.5*t)/(2*k-1), linewidth=0.5, color="gray")

# Se suman los armónicos y se representa la onda cuadrada resultante:
suma = 0
for k in range(1,9):
    suma = suma + (8/np.pi)*np.sin(2*np.pi*(2*k-1)*0.5*t)/(2*k-1)
plt.plot(t, suma, linewidth=1, color="red", label="Onda cuadrada resultante")

# Se define diversos aspectos del formato de la gráfica (título, leyenda, etc.):
plt.title("Onda cuadrada construida por síntesis aditiva \n a partir de sus ocho primeros armónicos")
plt.xlabel("Tiempo")
plt.ylabel("Amplitud")
plt.legend(loc="lower right")
plt.grid(True, which="both")
plt.axhline(y=0, color="k")
plt.ylim(-3, 3)

# Se exporta la gráfica a formato ".pgf" y ".pdf"
plt.savefig("SquareWave.pgf", format="pgf")
plt.savefig("SquareWave.pdf", dpi=400)
plt.clf()
```

Código 9.1: Código utilizado para generar la gráfica **??**

La inmensa mayoría de operaciones relacionadas con el formato del código se realizan

en el preámbulo del documento (cuyo código está comentado). En él se definen todos los parámetros necesarios para que el paquete `listings` se adapte a código escrito en Python (palabras clave, colores, comentarios, etc.). Si se desea incluir código de otro lenguaje de programación, es necesario modificar el preámbulo para que `listings` se adapte a dicho lenguaje (en la mayoría de casos se puede encontrar el código adaptado a cada lenguaje de programación en foros relacionados con LaTeX).

Los únicos parámetros que pueden modificarse en este punto del código son `firstnumber`, cuyo valor corresponde al número de la primera línea de código (en este caso se ha fijado en 1) y `breakindent`, que indica el ancho de la indentación al realizar un salto de línea.

El formato elegido para la bibliografía es IEEE , tanto para las referencias a lo largo del documento como para el apartado de bibliografía. El conjunto de operaciones realizadas para establecer el formato de la bibliografía se puede consultar en el preámbulo del documento (en el que se describen algunos de sus parámetros básicos como el contenido de las referencias, el número de autores por cita, etc.).

Citar una referencia es sencillo, basta con utilizar el comando `\cite` seguido del nombre de la referencia correspondiente (el nombre utilizado en el archivo *.bib*, que es esencial cargar en el directorio de trabajo y cuyas principales características pueden consultarse en `https://en.wikipedia.org/wiki/BibTeX`), por ejemplo:

- *The Art of Electronics* constituye un fantástico manual (plagado de ejemplos prácticos y explicaciones tangibles) para aprender electrónica, siendo su tercera edición la versión más completa [2].
- *The Loudspeaker Design Cookbook* [1] es probablemente la guía más completa en cuanto a acústica aplicada al diseño de sistemas de sonido, abarcando desde conceptos teóricos de electroacústica hasta planos para la construcción de sistemas de sonido caseros.
- *Les fous du son* [6] es un relato cuidadosamente escrito y documentado sobre la historia de los sintetizadores desde Edison hasta nuestros días, pasando por los inventos más inverosímiles como las Ondas Martenot o el Trautonium.
- En su artículo de 2003 [4], el co-fundador de Shazam describe el funcionamiento de su algoritmo de búsqueda para archivos de audio.

BIBLIOGRAFÍA

[1] V. Dickason, *The Loudspeaker Design Cookbook*, 7.ª ed. Audio Amateur Press, 2007, ISBN: 9781882580477.

[2] P. Horowitz y W. Hill, *The Art of Electronics*, 3.ª ed. Cambridge University Press, 2015, ISBN: 9780521809269.

[3] *Introduction to Altair in Python*, en, Section: Python, jul. de 2025. dirección: `https://www.geeksforgeeks.org/python/introduction-to-altair-in-python/` (visitado 07-04-2026).

[4] A. Wang, «An industrial strength audio search algorithm,» en *ISMIR 2003, 4th Symposium Conference on Music Information Retrieval*, 2003, págs. 7-13.

[5] *Why DuckDB*, en, 2026. dirección: `https://duckdb.org/why_duckdb.html` (visitado 07-04-2026).

[6] L. de Wilde, *Les fous du son*. Grasset, 2016, ISBN: 9782246859284.

ANEXOS

tantos como se considere necesario en funci´on de los aspectos concretos que se quieran especificar.

A. Primer anexo

Contenido del primer anexo (texto, tablas, figuras, códigos, etc.)

B. Segundo anexo

Contenido del segundo anexo (texto, tablas, figuras, códigos, etc.)

interwordspace: 3.91663pt

interwordstretch: 1.95831pt

emergencystretch: 35.24963pt